CP(M) 2026: Invigilated MiniZinc Assessment
Mock paper

This is a mock paper for practice and is not assessed. It has the same format, instructions and appendices as the Invigilated MiniZinc Assessment on Wednesday 11 November 2026, which is worth 30% of the course grade and is band marked, but the questions are different.

Instructions.

- Attempt all questions. The indicative weight of each question is given in its heading.
- Appendix A contains a constraint programming model for graph burning, and Appendix B contains an integer linear programming model for the same problem. Both are sample solutions to Task 3 of Lab 3: they use the declarations and output item of the Lab 3 stub, and minimise `turns`. Their constraints are labelled (A), (B), ... in comments, and the questions refer to them by these labels.
- Where you are asked to write MiniZinc, small syntax errors will not be penalised. The marks are for the modelling.

Background

All graphs are finite, simple and undirected, with vertex set $V = \{1, \ldots, n\}$. We write $d(u, v)$ for the distance between u and v, and $\mathrm{rad}(G)$ for the radius of a connected graph G.

Graph burning is as defined in Lab 3. At turn 0 every vertex is unburnt. At every turn $t \geq 1$, first every unburnt vertex adjacent to a burning vertex becomes burning, and then the fire-lighter chooses a vertex to set burning. A sequence $(x_1, \ldots, x_b)$ of choices is a *burning sequence* if every vertex is burning at the end of turn b, and the *burning number* $b(G)$ is the length of a shortest burning sequence. As shown in Lab 3, $(x_1, \ldots, x_b)$ is a burning sequence if and only if every vertex u satisfies $d(u, x_i) \leq b - i$ for some i.

Both models compute the same parameter `UB`, which is $\mathrm{rad}(G)+1$ if G is connected and n otherwise. Instances named in the questions are those of Lab 3. In particular, `path-16` is the path $1, 2, \ldots, 16$, for which $\texttt{UB} = 9$ and $b(G) = 4$, and we write K_2 for the graph with two vertices and one edge.

Question 1: the CP model (40%)

These questions concern the model in Appendix A.

1. Sketch a proof that constraints (D), (E) and (F) together ensure that, for every turn t in $0, \ldots,$ `UB` and every vertex v, `burnt[t, v]` is true exactly when v is burning at the end of turn t, when the vertex chosen at each turn t is the vertex v with `burn_turn[v]` $= t$, if any.
2. Suppose that constraint (H) is replaced by `burn_turn[v] = t -> not spread[t, v]`, that is, the second disjunct is deleted. Does the modified model still compute $b(G)$ for every graph G? Justify your answer, and if it does not, give a graph on which the modified model gives a wrong answer and explain what goes wrong.

Question 2: the ILP model (40%)

These questions concern the model in Appendix B.

1. On `path-16`, the model returns a solution in which `x[4, 3]`, `x[10, 2]`, `x[14, 1]` and `x[16, 0]` are 1 and every other entry of `x` is 0. Give the values of `y`, `turns` and `burn_turn[v]` for every vertex v, and the sequence printed by the output item. Using your values, verify constraint (E) for $u = 7$ and for $u = 13$.
2. Constraint (F) sets the value of `burn_turn`, using `UB` as a big-M constant. Explain in 1-2 sentences why (F) cannot simply be written as

   ```
   constraint forall(v in 1..n)(
     burn_turn[v] = sum(r in RADIUS)((turns - r) * x[v, r]));
   ```

   in an integer linear program, and why $M =$ `UB` is large enough.

Question 3: modifying the models (20%)

This question is about modifying the models in Appendices A and B to solve a related problem.

Say that the problem budget-k-burning is a modification of graph burning in which at each turn the fire-lighter may choose at most k vertices to set burning, where $k \geq 1$ is a parameter, and the fire spreads as before. Thus budget-1-burning has the same optimal value as classical graph burning, as implemented in the models in Appendices A and B. The budget-k-burning number of a graph is the minimum number of turns after which every vertex is burning.

- Describe the changes needed to each model, writing any new or changed constraints in MiniZinc. You may want to refer to line numbers.
- The budget-2-burning number of `path-16` is 3. Give a sequence of choices that burns `path-16` in three turns with at most two vertices chosen at each turn, and explain why two turns are not enough.

Appendix A: CP model

```
% Graph burning: CP sample solution (simulation model)

include "alldifferent_except_0.mzn";

int: n;                       % number of vertices
int: m;                       % number of edges

array[1..m] of int: from;     % one end of each edge
array[1..m] of int: to;       % the other end of each edge

% nbr[v] is the set of neighbours of v (a parameter, computed before solving)
array[1..n] of set of int: nbr =
  [ {to[e] | e in 1..m where from[e] = v} union {from[e] | e in 1..m where to[e] = v}
  | v in 1..n ];

% ball(v, r) is the set of vertices at distance at most r from v
function set of int: ball(int: v, int: r) =
  if r = 0 then {v}
  else let { set of int: B = ball(v, r-1) } in B union array_union([nbr[u] | u in B])
  endif;

% Upper bound on the number of turns. If G is connected, choosing a centre at
% turn 1 burns everything by turn rad(G) + 1. Otherwise choosing every vertex
% in turn certainly works, so n is a bound.
bool: connected = card(ball(1, n)) = n;
function int: ecc(int: v) = min([r | r in 0..n-1 where card(ball(v, r)) = n]);
int: UB = if connected then min([ecc(v) | v in 1..n]) + 1 else n endif;

% burn_turn[v] = t if v is chosen at turn t, and 0 if v is never chosen
array[1..n] of var 0..UB: burn_turn;

% the number of turns taken to burn the whole graph
var 1..UB: turns;

% burnt[t, v]: v is burning at the end of turn t
array[0..UB, 1..n] of var bool: burnt;

% spread[t, v]: v is burning after the spread in turn t, before the choice
array[1..UB, 1..n] of var bool: spread;

% --- The burning sequence ------------------------------------------------

% (A)
constraint alldifferent_except_0(burn_turn);

% (B)
constraint forall(v in 1..n)(burn_turn[v] <= turns);

% (C)
constraint count(v in 1..n)(burn_turn[v] > 0) = turns;

```

```
% --- Simulating the fire ------------------------------------------------

% (D)
constraint forall(v in 1..n)(not burnt[0, v]);

% (E)
constraint forall(t in 1..UB, v in 1..n)(
  spread[t, v] <-> (burnt[t-1, v] \/ exists(u in nbr[v])(burnt[t-1, u]))
);

% (F)
constraint forall(t in 1..UB, v in 1..n)(
  burnt[t, v] <-> (spread[t, v] \/ burn_turn[v] = t)
);

% (G)
constraint forall(v in 1..n)(burnt[turns, v]);

% --- Removing redundant choices -----------------------------------------

% (H)
constraint forall(t in 1..UB, v in 1..n)(
  burn_turn[v] = t -> (not spread[t, v] \/ forall(u in 1..n)(spread[t, u]))
);

% Fix turns first, trying small values first: this behaves like solving the
% decision problem for T = 1, 2, ... in order. Then choose the fires.
solve :: seq_search([
           int_search([turns], input_order, indomain_min),
           int_search(burn_turn, first_fail, indomain_max)])
      minimize turns;

output ["turns = \(turns);\n",
        "sequence = \([v | t in 1..fix(turns), v in 1..n where fix(burn_turn[v]) = t]);\n"];
```

Appendix B: ILP model

```
% Graph burning: ILP sample solution (covering model)
%
% A sequence (x_1, ..., x_b) burns G iff every vertex u has d(u, x_i) <= b - i
% for some i: the fire lit at turn i has radius b - i by the end of turn b.
% Every constraint below is a linear equation or inequality.

int: n;                       % number of vertices
int: m;                       % number of edges

array[1..m] of int: from;     % one end of each edge
array[1..m] of int: to;       % the other end of each edge

% nbr[v] is the set of neighbours of v (a parameter, computed before solving)
array[1..n] of set of int: nbr =
  [ {to[e] | e in 1..m where from[e] = v} union {from[e] | e in 1..m where to[e] = v}
  | v in 1..n ];

% ball(v, r) is the set of vertices at distance at most r from v
function set of int: ball(int: v, int: r) =
  if r = 0 then {v}
  else let { set of int: B = ball(v, r-1) } in B union array_union([nbr[u] | u in B])
  endif;

% Upper bound on the number of turns, as in the CP model
bool: connected = card(ball(1, n)) = n;
function int: ecc(int: v) = min([r | r in 0..n-1 where card(ball(v, r)) = n]);
int: UB = if connected then min([ecc(v) | v in 1..n]) + 1 else n endif;

% d[u, v] is the distance from u to v, capped at UB (larger distances never matter)
array[1..n, 1..n] of int: d =
  let { array[1..n, 0..UB] of set of int: B =
          array2d(1..n, 0..UB, [ball(v, r) | v in 1..n, r in 0..UB]) }
  in array2d(1..n, 1..n,
       [ min([r | r in 0..UB where u in B[v, r]] ++ [UB]) | u, v in 1..n ]);

% Radii a fire can have at the end of the last turn
set of int: RADIUS = 0..UB-1;

% x[v, r] = 1 if v is chosen at turn turns - r, so that its fire has radius r
% at the end of the last turn
array[1..n, RADIUS] of var 0..1: x;

% y[r] = 1 if some vertex is lit with radius r, that is, if r < turns
array[RADIUS] of var 0..1: y;

% the stub's variables
array[1..n] of var 0..UB: burn_turn;
var 1..UB: turns;

% --- The burning sequence ------------------------------------------------

```

```
% (A)
constraint forall(r in RADIUS)(sum(v in 1..n)(x[v, r]) = y[r]);

% (B)
constraint forall(r in 1..UB-1)(y[r] <= y[r-1]);

% (C)
constraint turns = sum(r in RADIUS)(y[r]);

% (D)
constraint forall(v in 1..n)(sum(r in RADIUS)(x[v, r]) <= 1);

% --- Covering -----------------------------------------------------------

% (E)
constraint forall(u in 1..n)(
  sum(v in 1..n, r in RADIUS where d[u, v] <= r)(x[v, r]) >= 1
);

% --- Recovering burn_turn for the output --------------------------------

% (F)
constraint forall(v in 1..n)(
  let { var int: chosen = sum(r in RADIUS)(x[v, r]);
        var int: radius = sum(r in RADIUS)(r * x[v, r]) } in
       burn_turn[v] <= UB * chosen
    /\ burn_turn[v] <= turns - radius + UB * (1 - chosen)
    /\ burn_turn[v] >= turns - radius - UB * (1 - chosen)
);

solve minimize turns;

output ["turns = \(turns);\n",
        "sequence = \([v | t in 1..fix(turns), v in 1..n where fix(burn_turn[v]) = t]);\n"];
```